# Adaptive Parallel Sorting System Using Dynamic Algorithm Selection

## Parallel and Distributing Computing (UCS645)

Sudeep (102483062)
Tejnoor Singh (102483089)
Vansh Sachdeva (102483066)

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology

May 2025

# Outline


1. Introduction
2. Problem Formulation
3. Methodology
4. System Design
5. Results
6. Conclusion

# Introduction

- **Background:** Sorting is fundamental for databases and analytics.
- **Problem:** Blind parallelism causes overhead for small datasets due to thread management and synchronization.
- **Solution:** An intelligent adaptive system that switches between serial and parallel merge sort based on input size $N$.

# Research Gaps & Objectives

## Research Gaps

- Absence of dynamic decision logic in standard libraries.
- Lack of integrated performance analysis (serial overhead vs. parallel gain).

## Objectives

- Implement an adaptive sorting system in C++.
- Parallelize Merge Sort using **OpenMP**.
- Maximize throughput and minimize resource idle time.

# Problem Formulation

The total execution time $T$ for input size $N$ and threads $p$ is modeled as:

$$T(N,p) = \begin{cases} T_{serial}(N) & \text{if } N < N_{threshold} \\ T_{parallel}(N,p) + O_{overhead} & \text{if } N \geq N_{threshold} \end{cases}$$

- $O_{overhead}$**:** Cost of thread creation and task scheduling.
- **Goal:** Minimize $T$ by identifying the optimal $N_{threshold}$ ($\approx 10^5$).

# Comparative Performance Analysis

The system evaluates performance by comparing three distinct sorting implementations:

- **Serial Merge Sort:** Baseline implementation running on a single CPU core.
- **Naive Parallel Merge Sort:** Employs parallelism for all recursive calls; demonstrates high overhead for small $N$.
- **Adaptive Parallel Sort:** Dynamically selects the optimal algorithm based on input size $N$ and $N_{threshold}$.

# Adaptive Logic Algorithm

**Require:** Array $A$, Size $N$

```
if N < 100000 then
    SerialMergeSort(A, 0, N − 1)
else
    #pragma omp parallel
    #pragma omp single
    ParallelMergeSort(A, 0, N − 1)
end if
```

# Parallelization Strategy

- **Level 1 (Adaptive):** Workload partitioning based on $N$.
- **Level 2 (OpenMP):** Task-based parallelism using `#pragma omp task`.
- **Optimization:** Threshold-based serial cutoff within recursion to prevent task explosion.

# System Design

- **sort/:** Core logic for Serial and Parallel Merge Sort.
- **logic/:** Adaptive decision module.
- **util/:** High-resolution timers and random array generators.
- **Key Decision:** Used **Tasks** instead of `parallel for` to better handle the recursive nature of the algorithm.

# Results & Performance

Table: Baseline Results ($N = 10^7$)

| Metric | Value |
|---|---|
| Serial Execution Time | $\sim$2.4 s |
| Parallel Time ($T = 8$) | $\sim$0.7 s |
| Speedup | $\sim$3.4x |
| Correctness Check | Passed |

- Near-linear speedup for large $N$.
- Serial mode remains superior for $N < 10^4$.

# Correctness Validation

## Critical Bugs Fixed

- **Race Condition:** Fixed by allocating thread-local temporary buffers for merging.
- **Synchronization:** Added `taskwait` to ensure sub-problems finish before merging.
- **Recursion:** Fixed off-by-one errors in midpoint calculations.

# Conclusion

- Adaptive logic eliminates parallel overhead for small inputs.
- Achieves significant speedup for large-scale data processing.
- Hybrid approach ensures responsiveness and scalability.
- **Future Work:** GPU acceleration using CUDA for even larger datasets.

# Thank You!